APP DEVELOPMENT I PROJECT REPORT

Trekking Management Application – V1

Name: Sahil Alam
Roll Number: 23f1002354
Email: 23f1002354@ds.study.iitm.ac.in
Course: App Development I (Project)

About Me

I am a student pursuing studies in Data Science with a strong interest in machine learning and deep learning. Through this project, I gained practical experience in designing and developing a role-based web application using Flask, SQLAlchemy, SQLite, and Bootstrap while implementing secure authentication, relational database management, and dynamic user interfaces.

0.1 Project Title

Trekking Management Application – V1

0.2 Problem Statement

Adventure organizations often rely on spreadsheets, phone calls, and manual coordination to manage trekking activities. This makes it difficult to assign trek guides, monitor participant registrations, manage slot availability, approve staff members, and track bookings efficiently. The objective of this project was to develop a centralized web-based Trekking Management Application that streamlines these operations by providing dedicated interfaces for the Admin, staff members, and participants while maintaining data consistency and role-based access control.

0.3 Approach

The application was developed using the Flask web framework following a modular architecture with separate route files for authentication, Admin, staff, and user functionalities. SQLite was used as the relational database, while SQLAlchemy ORM simplified database interactions and relationship management. Jinja2 templates and Bootstrap 5 were used to create a responsive and user-friendly interface.

The system implements three distinct user roles:

- **Admin:** Manages trekking events, assigns staff members, approves registrations, monitors bookings, and oversees the overall system.
- **Staff Members:** Manage their assigned treks by updating trek status, viewing participants, and handling booking cancellations when required.
- **Users:** Browse available treks, make bookings, cancel bookings before the trek begins, maintain their profile, and view their trekking history.

0.3.1 Role-Based Session Authorization

To secure application resources, stateful authentication was implemented using the client-side Flask `session` dictionary. Upon successful credential verification at the login gateway, user-specific data (including `user_id`, `role`, and `username`) is stored directly in the session cookie.

For security enforcement across routes, the system directly checks whether the session contains a valid `user_id` to verify if a user is logged in. To enforce role-based access control, the system checks the role stored in the session (e.g., verifying if the active role matches the required permission, such as `'admin'`, `'staff'`, or `'user'`). If a user attempts to access an endpoint that is restricted for their role (such as a standard user attempting to access path routes intended only for the Admin), the system blocks access and handles the request accordingly, ensuring isolation between the Admin, Staff, and general User contexts.

Several validation rules were also implemented to ensure data integrity, including duplicate booking prevention, date sequence constraints, slot capacity checks, valid trek status transitions, and automatic booking status updates upon trek completion.

0.4 AI / LLM Declaration

This project was developed primarily by me. During the development process, ChatGPT (GPT-5) was used as an AI assistant for brainstorming ideas, debugging Flask and SQLAlchemy code, refining database queries, improving validation logic, and providing suggestions for improving the user interface. Approximately 20–25% of the development involved AI-assisted guidance.

All architectural decisions, database design, business logic, testing, integration, and final implementation were completed manually by me. Every piece of generated code was reviewed, modified where necessary, tested, and integrated into the application before submission.

0.5 Frameworks and Libraries Used

Framework / Library	Purpose
Flask	Core backend web framework and controller manager
SQLAlchemy	Object Relational Mapping (ORM) database interactions
SQLite	Local relational database management system
Jinja2	Dynamic HTML template rendering and layout generation
Bootstrap 5	Responsive frontend styling and user interface design
HTML5 & CSS3	Structural and custom layout design
Werkzeug	Secure password hashing and secure image upload handling
Datetime	Date validation, scheduling, and booking management
OS Module	File system operations, image storage, and file management

0.6 Database Design (ER Diagram)

The application follows a highly structured relational database design consisting of three core entities:

0.6.1 Tables and Schema Details

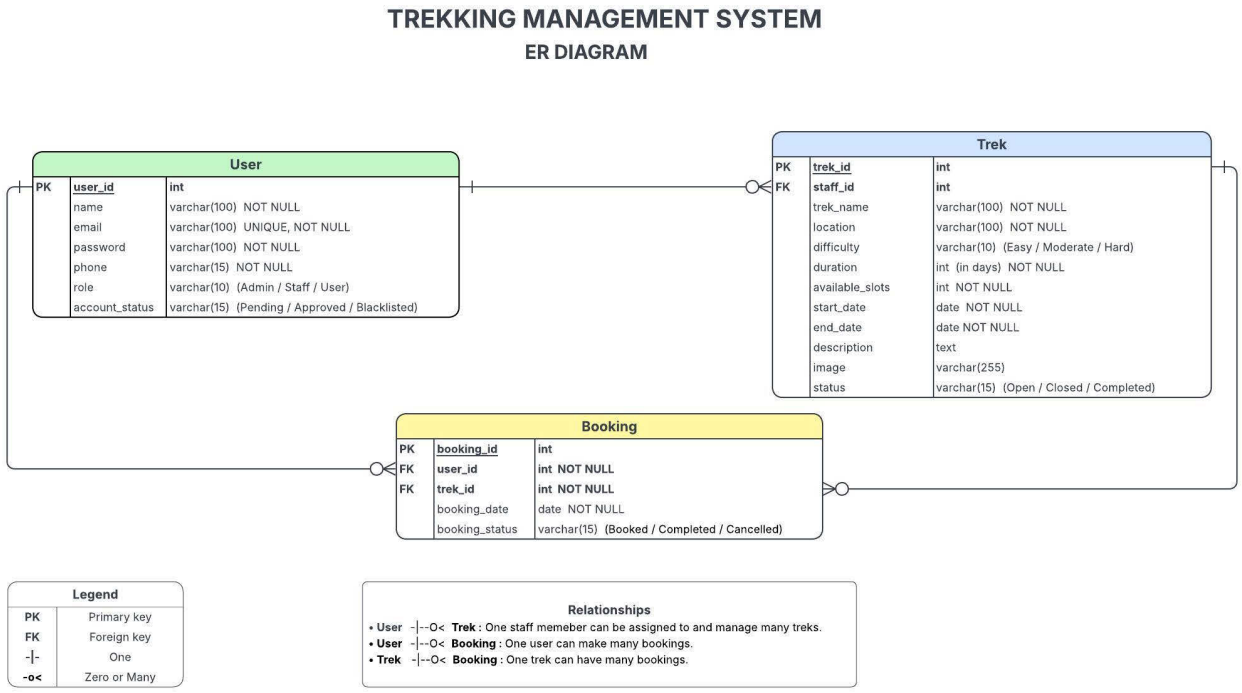
1. **User:** Stores system actors including the Admin, staff members, and participants. Key attributes include `user_id`, `name`, `email`, `password`, `phone`, `role`, and `account_status`.

2. **Trek:** Stores trek configurations managed by the Admin and staff. Key attributes include `trek_id`, `staff_id`, `trek_name`, `location`, `difficulty`, `duration`, `available_slots`, `start_date`, `end_date`, `description`, `image`, and `status`.
3. **Booking:** Acts as a junction table tracking user registrations for treks. Key attributes include `booking_id`, `user_id`, `trek_id`, `booking_date`, and `booking_status`.

0.6.2 Relationships

- **User to Trek (One-to-Many):** One staff member can be assigned to and manage zero or many treks.
- **User to Booking (One-to-Many):** One participant user can place zero or many trek bookings.
- **Trek to Booking (One-to-Many):** One trekking event can contain zero or many participant bookings.

0.6.3 ER Diagram



0.7 API & Application Resource Endpoints

Endpoint	Method	Description
`/login`	GET, POST	User authentication login portal (creates Flask session)
`/register`	GET, POST	User account registration
`/logout`	GET	Destroys active session cookie context
`/admin`	GET	Admin landing control dashboard
`/admin/treks`	GET	Master control system for the Admin to configure treks
`/admin/staff`	GET	Manage and verify staff member profiles
`/admin/users`	GET	Oversight panel for general user records
`/admin/bookings`	GET	System-wide booking audit ledger
`/staff`	GET	Staff personnel overview dashboard
`/staff/treks`	GET	Lists treks currently assigned to the logged-in staff
`/staff/profile`	GET, POST	Read and update staff contact profile details
`/user`	GET	General participant dashboard portal
`/user/treks`	GET	Browse all available, upcoming, and open treks
`/user/bookings`	GET	Active participant tracking for user bookings
`/user/history`	GET	Historical log of completed trek adventures
`/user/profile`	GET, POST	Update participant profile parameters

0.8 Presentation Video

0.8.1 Google Drive Link

Please access the project presentation video via the following active shareable link:

https://drive.google.com/file/d/1Gr5Ct9bpBsKHVi3vLVjdIYVPOUPBrmAK/view?usp=sharing

0.9 Conclusion

The Trekking Management Application – V1 successfully addresses the challenges of manual trek tracking by providing a centralized, role-based web application. The system enables the Admin to efficiently manage treks, staff members, users, and bookings, while allowing staff to oversee assigned treks and participants. Users can conveniently browse available treks, make bookings, manage their profile, and view their trekking history.

The application enforces strict data integrity constraints through comprehensive validation rules, secure session-based role authorization, clean relational database normalization, and automated business logic like slot management and dynamic status updates. Overall, the project demonstrates a robust and practical implementation of Flask, SQLAlchemy, SQLite, Jinja2, and Bootstrap 5 in developing a complete database-driven web solution.